

فصل ۱۰

حافظه‌ی مجازی

در فصل ۹، راهبردهای مختلف مدیریت حافظه را که در سیستم‌های کامپیوتری استفاده می‌شوند بحث کردیم. تمامی این راهبردها هدف یکسانی دارند: نگه‌داری همزمان پردازه‌های متعدد در حافظه به منظور داشتن چندبرنامگی. با این وجود، در تمامی این راهبردها لازم است که کل پردازه پیش از اجرا در حافظه قرار بگیرد. حافظه‌ی مجازی تکنیکی است که امکان اجرای پردازه‌هایی را فراهم می‌کند که به طور کامل در حافظه قرار ندارند. یکی از مزایای مهم این طرح آن است که برنامه‌ها می‌توانند بزرگ‌تر از حافظه‌ی فیزیکی باشند. به علاوه، حافظه‌ی مجازی مدل بینهایت بزرگ آرایه‌ی یکنواخت حافظه‌ی اصلی را با تفکیک حافظه‌ی منطقی دید برنامه‌نویس از حافظه‌ی فیزیکی فراهم می‌کند. این تکنیک، برنامه‌نویسان را از نگرانی محدودیت‌های ذخیره‌سازی حافظه رها می‌سازد. حافظه‌ی مجازی، امکان اشتراک فایل‌ها و کتابخانه‌ها، و پیاده‌سازی حافظه‌ی مشترک را نیز به‌سادگی فراهم می‌نماید. به علاوه، راهکار کارآمد و مؤثری هم برای ایجاد پردازه مهیا می‌کند. با این همه، پیاده‌سازی حافظه‌ی مجازی چندان هم ساده نبوده و در صورتی که با بی‌دقتی مورد استفاده قرار بگیرد، ممکن است کارایی را به طور چشم‌گیری کاهش بدهد. ما در این فصل، یک مرور مفصلی از حافظه‌ی مجازی را ارائه داده، چگونگی پیاده‌سازی آن را بررسی کرده، و پیچیدگی و منافع آن را مطرح می‌کنیم.

اهداف فصل

- تعریف حافظه‌ی مجازی و تبیین منافع آن.
- نمایش چگونگی بارگذاری صفحات به‌داخل حافظه با استفاده از صفحه‌بندی نیازی.
- به‌کارگیری الگوریتم‌های جایگزینی صفحه‌ی FIFO، بهینه، و LRU.
- تبیین مجموعه‌ی کاری یک پردازه، و تشریح چگونگی ارتباط آن با همسایگی برنامه.
- تبیین چگونگی مدیریت حافظه‌ی مجازی لینوکس، ویندوز ۱۰، و سولاریس.
- طراحی یک شبیه‌ساز مدیر حافظه‌ی مجازی به زبان برنامه‌نویسی C.

۱۰-۱ پس‌زمینه

الگوریتم‌های مدیریت حافظه‌ای که در فصل ۹ مطرح شدند به دلیل یک نیاز اساسی لازمند:

دستورالعمل‌های اجرایی بایستی در حافظه‌ی فیزیکی قرار بگیرند. اولین رویکرد برآوردن این نیاز آن است که کل فضای آدرس منطقی در حافظه‌ی فیزیکی مستقر شود. اتصال پویا می‌تواند این محدودیت را خفیف‌تر کند، اما عموماً مستلزم محتاطکاری خاصی بوده و کار بیشتر برنامه‌نویس را می‌طلبد.

به نظر می‌رسد این نیاز که بایستی دستورالعمل‌ها برای اجرا در حافظه‌ی فیزیکی قرار بگیرند هم ضروری و هم مستدل است؛ اما چندان هم مطلوب نیست، چرا که اندازه‌ی یک برنامه را به اندازه‌ی حافظه‌ی فیزیکی محدود می‌کند. در حقیقت، یک بررسی از برنامه‌های واقعی به ما نشان می‌دهد که در بسیاری از موارد، کل برنامه مورد نیاز نیست. برای نمونه، موارد زیر را ملاحظه کنید:

- برنامه‌ها اغلب کدهایی برای مواجهه با شرایط خطای غیرمعمول دارند. چون این خطاها، حتی اگر در عمل هم اتفاق بیفتند نادر هستند، چنین کدی تقریباً هرگز اجرا نخواهد داشت.
- اغلب به آرایه‌ها، لیست‌ها و جداول، حافظه‌ی بیشتری از نیاز آنها تخصیص می‌یابد. یک آرایه می‌تواند با تعداد عناصر ۱۰۰ در ۱۰۰ اعلان شود، به رغم این که به ندرت از ۱۰ در ۱۰ عناصر خود استفاده می‌نماید.
- گزینه‌ها و امکانات خاص یک برنامه شاید به ندرت مورد استفاده قرار بگیرند. برای نمونه، روال‌های کامپیوترهای دولت ایالات متحده که برای موازنه‌ی بودجه به کار گرفته می‌شوند سال‌هاست که استفاده نشده‌اند.

حتی در مواردی هم که برنامه تماماً مورد نیاز است، امکان دارد که کل برنامه همزمان لازم نشود.

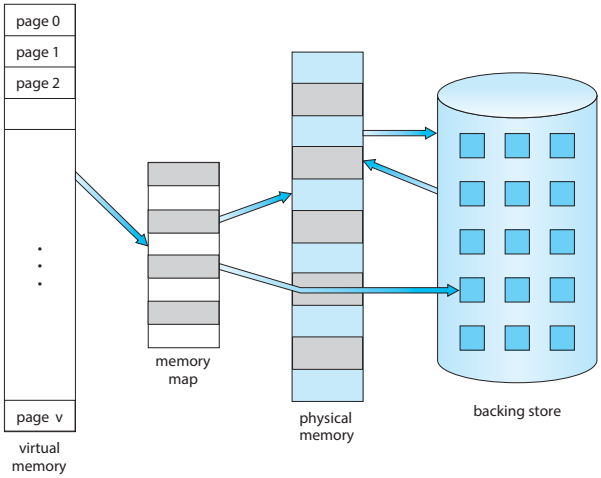
قابلیت اجرای برنامه‌ای که فقط بخشی از آن در حافظه قرار دارد، منافع زیادی به دنبال دارد که عبارتند از:

- برنامه دیگر به حجم موجود حافظه‌ی فیزیکی محدود نخواهد بود. کاربران قادر خواهند بود برنامه‌هایی برای یک فضای آدرس مجازی فوق‌العاده بزرگ بنویسند که کار برنامه‌نویسی هم آسان‌تر می‌شود.
- چون هر برنامه می‌تواند حافظه‌ی کمتری بگیرد، برنامه‌های بیشتری می‌توانند همزمان اجرا شوند که افزایش بهره‌وری پردازنده و بازده را در پی دارد، بی‌آنکه افزایشی در زمان پاسخ یا زمان گردش به بار آورد.
- برای بارگذاری یا مبادله‌ی بخش‌هایی از برنامه‌ها به یا از حافظه، I/O کمتری لازم خواهد شد، لذا هر برنامه با سرعت بیشتری اجرا خواهند شد.

بنابراین، اجرای برنامه‌ای که به طور کامل در داخل حافظه قرار ندارد به نفع هر دوی سیستم و کاربران آن بود.

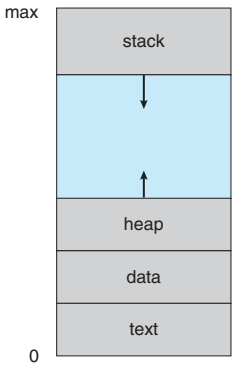
حافظه‌ی مجازی مشمول تفکیک تصور حافظه‌ی منطقی توسعه‌دهندگان از حافظه‌ی فیزیکی است. با این تفکیک حتی زمانی که حافظه‌ی فیزیکی کوچکتری هم وجود دارد باز هم حافظه‌ی مجازی بینهایت بزرگی برای برنامه‌نویسان فراهم می‌شود (شکل ۱۰-۱). حافظه‌ی مجازی کار

برنامه‌نویسی را بسیار آسان‌تر می‌سازد، چون دیگر برنامه‌نویس لازم نیست که نگران حجم حافظه‌ی فیزیکی موجود باشد؛ لذا به جای آن می‌تواند بیشتر بر روی برنامه‌نویسی مسئله‌ای که قرار است حل شود تمرکز کند.



شکل ۱۰-۱ دیاگرام نمایش بزرگی حافظه‌ی مجازی در مقایسه با حافظه فیزیکی.

فضای آدرس مجازی یک پردازه به دید منطقی (یا مجازی) چگونگی ذخیره‌سازی پردازه در حافظه مربوط می‌شود. نوعاً، این دیدگاه وجود دارد که پردازه از یک آدرس منطقی ویژه‌ای—برفرض، آدرس ۰—شروع می‌شود و همانند نمایش شکل ۱۰-۲، به صورت همجوار در حافظه قرار می‌گیرد. هرچند از فصل ۹ به یاد دارید که حافظه‌ی فیزیکی در حقیقت می‌تواند به صورت قاب‌صفحات سازماندهی شود و قاب‌های تخصیص یافته به یک پردازه ممکن است همجوار نباشند. این مسؤولیت واحد مدیریت حافظه (MMU) است که صفحات منطقی را به قاب‌صفحات فیزیکی حافظه نگاشت بدهد.



شکل ۱۰-۲ فضای آدرس مجازی یک پردازه در داخل حافظه.

توجه داشته باشید که در شکل ۱۰-۲، به heap، که برای تخصیص پویای حافظه استفاده می‌شود، اجازه می‌دهیم تا در حافظه به سمت بالا رشد نماید. با تشابهی، این امکان را برای پشته فراهم می‌کنیم که با تداوم فراخونی توابع به سمت پایین حافظه رشد داشته باشد. فضای خالی بزرگی (یا حفره‌ای) که بین heap و پشته وجود دارد بخشی از فضای آدرس مجازی محسوب می‌شود، ولی تنها در صورتی صفحات فیزیکی واقعی را لازم خواهد داشت که heap یا پشته رشد داشته باشند. فضاهای آدرس مجازی که حفره‌هایی را در بر دارند به فضاهای آدرس **خلوت**[1] معروفند. استفاده از یک فضای آدرس خلوت سودآور است چرا که حفره‌ها می‌توانند با رشد قطعات پشته یا heap یا با اتصال کتابخانه‌های پویا (یا احتمالاً سایر اشیاء مشترک) در طی اجرای برنامه پر شوند.

حافظه‌ی مجازی علاوه بر تفکیک حافظه‌ی منطقی از حافظه‌ی فیزیکی، امکان اشتراک فایل‌ها و حافظه را میان دو یا چند پردازه از طریق اشتراک صفحه (بخش ۹-۳-۴) فراهم می‌کند. این امکان منافع زیر را به دنبال دارد:

- کتابخانه‌های سیستمی، همچون کتابخانه‌ی استاندارد C می‌توانند توسط چندین پردازه از طریق نگاشت شئ مشترک به یک فضای آدرس مجازی به اشتراک گذاشته شوند. اگرچه هر یک از پردازه‌ها کتابخانه‌ها را بخشی از فضای آدرس مجازی خود می‌پندارند، صفحات واقعی این کتابخانه‌ها که مقیم حافظه‌ی فیزیکی می‌باشند توسط پردازه‌ها به اشتراک گذاشته شده‌اند (شکل ۱۰-۳). کتابخانه‌ای که به فضای آدرس هر یک از پردازه‌ها متصل می‌شود، نوعاً نگاشت فقط‌خواندنی دارد.
- به طور مشابه، پردازه‌ها می‌توانند حافظه را به اشتراک بگیرند. از فصل ۳ به یاد دارید که دو یا چند پردازه می‌توانند با استفاده از حافظه‌ی مشترک با همدیگر در ارتباط باشند. حافظه‌ی مجازی این امکان را به یک پردازه می‌دهد که ناحیه‌ای از حافظه را برای اشتراک با پردازه‌های دیگر ایجاد کند. پردازه‌هایی که این ناحیه را به اشتراک می‌گیرند آن را بخشی از فضای آدرس مجازی خودشان تصور می‌کنند، حال آنکه صفحات فیزیکی واقعی، همچنان که در شکل ۱۰-۳ نشان داده شده‌اند، مشترک هستند.
- صفحات می‌توانند در طی ایجاد پردازه با فراخوانی سیستمی ()fork به اشتراک گذاشته شوند، از این رو ایجاد پردازه تسریع می‌یابد.

این منافع—و سایر مزایای—حافظه‌ی مجازی را با جزئیات بیشتری در ادامه‌ی همین فصل مرور می‌کنیم. هرچند که، در ابتدا پیاده‌سازی حافظه‌ی مجازی را از طریق صفحه‌بندی نیازی بحث می‌کنیم.

۱۰-۲ صفحه‌بندی نیازی

چگونگی بارگذاری یک برنامه‌ی اجرایی را از حافظه‌ی ثانویه به حافظه در نظر بگیرید. یکی از

1. Sparse